

MARAN SUITE SYSTEM — Especificaciones de Desarrollo: Módulo Restaurante

Fecha: Marzo 2026

Módulo: Restaurante

Prioridad: Alta — Cierre operativo antes de migración a PostgreSQL

Contexto general

El módulo de Restaurante ya tiene estructura completa y funcional: plano de mesas con dos sectores (Bodega y Moneda), pedidos/comandas, menú con categorías, cierre con comprobante y método de pago, reservas de mesa, y recetas con costeo. Las siguientes tareas completan el módulo con los flujos operativos que faltan.

Las tareas están ordenadas de menor a mayor complejidad. Ejecutar en orden.

TAREA 1 — Mozo asignado al abrir una mesa

Prioridad: Alta | **Complejidad:** Baja | **Área:** Backend + Frontend

Descripción

Al abrir una mesa (iniciar un pedido), el sistema debe solicitar el nombre del mozo que está atendiendo esa mesa. Este dato debe quedar visible en el folio y en la vista del plano de mesas.

Cambios en el schema (`shared/schema.ts`)

Tabla `restaurantOrders` — **agregar campo:**

```
waiterName: text("waiter_name"), // Nombre del mozo (texto libre)
```

Cambios en el backend (`server/routes.ts`)

POST `/api/restaurant/orders` — Aceptar y guardar el campo `waiterName` .

GET `/api/restaurant/orders` — Incluir `waiterName` en la respuesta.

Cambios en el frontend (`client/src/pages/restaurant.tsx`)

Modal de apertura de mesa:

- Agregar campo de texto "Mozo" antes del campo de comensales.
- Campo obligatorio — no permitir abrir la mesa sin ingresar el nombre.
- Placeholder: "Nombre del mozo..."

Vista del plano de mesas:

- En las mesas ocupadas (color ámbar), mostrar debajo del número de mesa el nombre del mozo en texto pequeño.
- Ejemplo: "Mesa 5 — Juan"

Folio/ticket de cierre:

- Incluir el nombre del mozo en el encabezado del folio.

TAREA 2 — Áreas sin mesas: Room Service, Delivery, Solarium, SPA

Prioridad: Alta | **Complejidad:** Media | **Área:** Backend + Frontend

Descripción

Agregar nuevas áreas de servicio que no tienen plano de mesas físico pero sí generan pedidos y cuentas: Room Service, Delivery, Solarium y SPA. Todas comparten el mismo menú del restaurante. En lugar de un plano, se gestionan como una lista de órdenes activas.

Cambios en el schema (`shared/schema.ts`)

Tabla `restaurantAreas` — agregar campo:

```
hasTables: boolean("has_tables").notNull().default(true),  
// false para áreas sin mesas físicas (Room Service, Delivery, Solarium, SPA)
```

Tabla `restaurantOrders` — agregar campo:

```
orderLabel: text("order_label"),  
// Para áreas sin mesa: identifica el pedido (ej: "Hab. 302", "Delivery - García")
```

Datos seed a agregar en `server/storage.ts`

Agregar las siguientes áreas con `hasTables: false`:

- Room Service
- Delivery
- Solarium
- SPA

Cambios en el backend (`server/routes.ts`)

Sin cambios de endpoints. Los pedidos de estas áreas usan los mismos endpoints existentes, solo cambia el `areaId` y el `orderLabel` en lugar de `tableId`.

Para áreas sin mesas, `tableId` puede ser `null`. Validar que si `hasTables = false` entonces `tableId` es opcional.

Cambios en el frontend (`client/src/pages/restaurant.tsx`)

Navegación entre áreas:

- El selector de área ya existe (tabs o dropdown). Agregar las nuevas áreas.
- Cuando el área seleccionada tiene `hasTables: false`, mostrar la vista de "Órdenes directas" en lugar del plano de mesas.

Vista de órdenes directas (para áreas sin mesas):

- Lista de órdenes activas del área, ordenadas por hora de apertura.
- Cada orden muestra: etiqueta (`orderLabel`), mozo, hora de apertura, cantidad de ítems, total parcial, estado.
- Botón "Nueva orden" → abre modal con:
 - Campo "Identificación del pedido" (texto libre, obligatorio). Ejemplos por área:
 - Room Service: "Habitación 302"
 - Delivery: "Cliente García"
 - Solarium: "Reposera 7"
 - SPA: "Cabina 2 - Turno 10:00"
 - Campo "Mozo / responsable" (texto libre, obligatorio).

- Campo "Comensales" (opcional para estas áreas).
 - Al abrir una orden, el flujo de carga de ítems y cierre es idéntico al de una mesa normal.
 - El cierre incluye los mismos métodos de pago, tipos de comprobante y opción de cargo a habitación.
-

TAREA 3 — Cursos / Pasos en la comanda

Prioridad: Media | **Complejidad:** Media | **Área:** Backend + Frontend

Descripción

Implementar el sistema de "cursos" o "pasos" en los pedidos. Cada ítem de la comanda pertenece a un curso (Entradas, Principales, Postres). La comanda se envía toda junta pero solo el curso activo está visible para la cocina. El mozo avanza manualmente al siguiente curso cuando los platos del curso anterior fueron entregados.

Lógica de cursos

- Curso 1: Entradas
- Curso 2: Platos Principales
- Curso 3: Postres

Al enviar la comanda, todos los ítems se guardan pero solo los del curso 1 quedan en estado `pending` (visibles para cocina). Los ítems de cursos posteriores quedan en estado `waiting_course`.

Cuando el mozo avanza el curso, los ítems del siguiente curso pasan de `waiting_course` a `pending`.

Cambios en el schema (`shared/schema.ts`)

Tabla `orderItems` — agregar campo:

```
course: integer("course").notNull().default(1),  
// 1 = Entradas, 2 = Principales, 3 = Postres
```

Tabla `restaurantOrders` — agregar campo:

```
activeCourse: integer("active_course").notNull().default(1),
```

```
// Curso actualmente activo en esta mesa
```

Actualizar enum de estados de `orderItems` : Agregar estado: `waiting_course` (ítem cargado pero en espera de que se active su curso).

Cambios en el backend (`server/routes.ts`)

POST `/api/restaurant/order-items` — Al agregar un ítem, setear automáticamente:

- Si `course <= order.activeCourse` → estado `pending` .
- Si `course > order.activeCourse` → estado `waiting_course` .

Nuevo endpoint:

```
POST /api/restaurant/orders/:id/advance-course
```

- Incrementa `activeCourse` en 1 (máximo 3).
- Cambia todos los ítems con `course = nuevo activeCourse` y estado `waiting_course` a `pending` .
- Retorna el nuevo `activeCourse` y la cantidad de ítems activados.

Cambios en el frontend (`client/src/pages/restaurant.tsx`)

Selector de curso al agregar ítems:

- Al agregar un ítem a la comanda, mostrar selector de curso: Entradas / Principales / Postres.
- El curso activo de la mesa está preseleccionado por defecto, pero el mozo puede asignar un ítem a cualquier curso.
- Los ítems en `waiting_course` se muestran en la comanda con un ícono de reloj o color diferente, indicando que están "en espera".

Botón "Siguiente curso" en la mesa:

- Visible en el folio/comanda de la mesa cuando hay ítems en `waiting_course` para el próximo curso.
- Texto: "Activar curso 2 — Platos Principales" (dinámico según el curso siguiente).
- Al hacer clic, muestra confirmación: *"¿Activar los Platos Principales? Se enviarán X ítems a cocina."*

- Tras confirmar, los ítems del siguiente curso pasan a `pending` y el botón actualiza al curso siguiente.

Indicador de curso activo:

- En la cabecera del folio de la mesa, mostrar el curso activo: "Curso activo: Entradas" / "Platos Principales" / "Postres".

TAREA 4 — División de cuenta en partes iguales

Prioridad: Media | **Complejidad:** Media | **Área:** Frontend

Descripción

Al momento del cierre de una mesa, agregar la opción de dividir la cuenta en partes iguales entre N personas. Cada parte genera su propio comprobante y método de pago independiente.

Lógica de negocio

- El total de la mesa se divide en N partes iguales (redondeo hacia arriba en la última parte para cubrir diferencias de centavos).
- Cada parte puede pagarse con un método de pago diferente y generar un comprobante distinto.
- Una vez que todas las partes están pagadas, la mesa queda saldada y se puede cerrar.

Cambios en el schema (`shared/schema.ts`)

Nueva tabla `orderSplits` :

```
export const orderSplits = pgTable("order_splits", {
  id: serial("id").primaryKey(),
  orderId: integer("order_id").notNull().references(() => restaurantOrders.id),
  splitNumber: integer("split_number").notNull(), // 1, 2, 3... (número de parte)
  amount: numeric("amount", { precision: 10, scale: 2 }).notNull(), // monto de e
  method: text("method"), // método de pago al cerrar esta parte
  receiptType: text("receipt_type"), // tipo de comprobante
  isPaid: boolean("is_paid").default(false),
  paidAt: timestamp("paid_at"),
  createdAt: timestamp("created_at").defaultNow(),
});
```

Cambios en el backend (`server/routes.ts`)

POST `/api/restaurant/orders/:id/split` → Crear división (body: { parts: N })
GET `/api/restaurant/orders/:id/split` → Ver partes de la división
PATCH `/api/restaurant/orders/:id/split/:splitId` → Pagar una parte (body: { method

Lógica del endpoint POST `/split` :

- Solo disponible si la orden no está cerrada y no tiene una división activa.
- Divide el total en `parts` partes iguales. La última parte absorbe el redondeo.
- Crea N registros en `orderSplits`.

Lógica del endpoint PATCH `/split/:splitId` :

- Marca la parte como pagada con el método y comprobante indicados.
- Si todas las partes están pagadas, cierra automáticamente la orden.

Cambios en el frontend (`client/src/pages/restaurant.tsx`)

En el modal de cierre de mesa, agregar botón "Dividir cuenta":

Al hacer clic en "Dividir cuenta":

- Mostrar input: "¿En cuántas partes?" con botones rápidos: 2 / 3 / 4 / o número libre.
- Mostrar preview de la división: "3 partes de \$X cada una".
- Botón "Confirmar división".

Vista de división activa:

- Una vez confirmada, el modal de cierre muestra las N partes en lista.
- Cada parte tiene: número de parte, monto, estado (pendiente / pagada), y botón "Cobrar esta parte" que abre el selector de método de pago y comprobante para esa parte específica.
- Las partes ya pagadas se muestran con tilde verde y el método usado.
- Cuando todas las partes están pagadas, la mesa se cierra automáticamente.

Cancelar división:

- Mientras haya partes sin pagar, mostrar botón "Cancelar división" que elimina los splits y vuelve al cierre normal.
-

Notas para Replit

- Ejecutar las tareas **en el orden indicado** (1 → 4).
- La Tarea 3 (cursos) requiere actualizar el MemStorage para manejar el nuevo campo `course` en `orderItems` y `activeCourse` en `restaurantOrders`.
- El estado `waiting_course` debe agregarse al enum/validación de estados de `orderItems` en `shared/schema.ts`.
- Para la Tarea 2, las nuevas áreas (Room Service, Delivery, Solarium, SPA) deben agregarse en los datos seed de `server/storage.ts` con `hasTables: false`.
- El campo `tableId` en `restaurantOrders` debe ser opcional (`nullable`) para soportar órdenes de áreas sin mesas.
- Mantener la nomenclatura existente (camelCase en TypeScript, snake_case en SQL).
- Los textos de interfaz deben estar en español, consistente con el resto del sistema.

Documento generado para el proyecto Maran Suite System — Módulo Restaurante
Próximo módulo a especificar: según prioridad operativa acordada